

INTRODUÇÃO À ANÁLISE DE SISTEMAS

Lei de Moore (Moore, 1965), proposta por um dos fundadores da Intel, Gordon Moore, que observou que a quantidade de transistores em um processador duplicava a cada 12 meses.

alguns dos reflexos da Lei de Moore:

- Diminuição do custo de produção: com o baixo custo de produção dos computadores, foi possível reduzir o preço final e, conseqüentemente, houve um aumento nas vendas e na produção em escala dos equipamentos.
- Diminuição do consumo de energia: os computadores passaram a consumir menos energia, tornando-se cada vez mais atrativos a um público cada vez mais crescente.
- Computadores menores: com componentes menores, os computadores também reduziram de tamanho. Passamos a não ter mais a necessidade de um local físico específico e especial para o computador.
- Computadores mais confiáveis: com um maior número de transistores em uma pastilha, diminuiu a necessidade de conexões entre pastilhas.

Sistemas de informação x software

Sistema de informação é a composição de informações (ou dados), pessoas, processos e tecnologias que tem por objetivo apoiar ou melhorar o processo de negócio de uma corporação.

Sistemas de informação tem foco na informação gerada em um processo organizacional, de tal forma a originar valor a esse processo.

Tecnologia de sistemas de informação, estamos falando de toda plataforma tecnológica que dá suporte ao processo de negócio, como infraestrutura de rede, servidores, computadores, sistemas operacionais e sistemas de software

Na plataforma tecnológica, deveríamos nos atentar a detalhes como como e onde essa informação seria armazenada e qual o melhor projeto de rede para suportar o tráfego dessa informação.

Projetar um sistema de software compreende termos a noção de que ele é um componente de um sistema de informação

Diversas foram as abordagens e técnicas criadas para modelar sistemas de software até chegarmos à abordagem foco deste livro: orientação a objetos. Por ora, entenda que a orientação a objetos é uma forma de se projetar um sistema de software.

Um sistema de software também pode ser identificado como o conjunto de vários componentes que, quando executados, produzem um resultado previamente desejado.

A Engenharia de Software tem por objetivo apoiar o desenvolvimento de software fornecendo técnicas para especificar, projetar, manter e gerir um sistema

Ela é baseada em três pilares:

- Métodos: provêm um conjunto de atividades com ênfase no como fazer para desenvolver um sistema de software.
- Ferramentas: mecanismos que dão suporte à execução das atividades descritas no método.
- Processos: proveem o elo entre métodos e ferramentas. Definem o quando e onde fazer, indicando a sequência de execução das atividades, quais ferramentas utilizar e quem deve ser o responsável pela atividade.

PROJETO DE DESENVOLVIMENTO

Processo de desenvolvimento de software

São um conjunto de atividades executadas em uma determinada sequência. Esse conjunto de atividades também pode ser chamado de etapas da Engenharia de Software

Especificação de software - Fase na qual são definidas e especificadas as funcionalidades do software, seus requisitos e suas regras de execução

Projeto e implementação de software - Fase na qual os requisitos são transformados em sistema de software. Quando são projetados e construídos os componentes de software e suas integrações

Validação de software - Fase na qual o software é validado frente aos requisitos especificados

Evolução de software - Fase na qual são traçadas estratégias para que o sistema de software possa ser mantido e evoluído na medida

Processo de desenvolvimento de software. Cada atividade possui:

- Pré e pós-condições: qual é o cenário pré e pós-execução de uma atividade. Por exemplo: para validação de um software, é necessário que este esteja implementado, após execução da validação, espera-se que o software esteja validado.

- Papéis: quem é o responsável pela execução da tarefa. Por exemplo: o analista de testes é responsável pela validação.
- Produtos: também chamados de artefatos, é o que é produzido como saída de uma determinada atividade. Por exemplo: espera-se como artefato de saída da atividade de especificação de software um documento contendo a descrição detalhada de todas as necessidades do negócio.

Modelos de processo de software

Modelo cascata

Também chamado de waterfall ou também citado na literatura como ciclo de vida clássico, o modelo cascata é composto de cinco atividades: Análise de requisitos, Projeto, Codificação, Testes, Implantação e manutenção.

No modelo cascata, as atividades são executadas de forma sequencial. Assim, uma atividade não é iniciada até que sua predecessora seja completamente finalizada. Isso nos permite afirmar, por exemplo, que a construção não é iniciada até que seja finalizada a arquitetura do sistema de software na fase de projeto, que, por sua vez, não é iniciada até que todos os requisitos sejam levantados, documentados e aprovados pelo usuário, é muito eficaz em projetos nos quais temos o escopo e as necessidades bem definidas e com poucas chances de mudança

Modelo incremental

O modelo incremental, também chamado de iterativo. No modelo iterativo, as atividades e a sequência de execução são exatamente as mesmas do modelo cascata. A diferença é que no modelo incremental não é necessário que todos os requisitos estejam mapeados para se iniciar o ciclo de desenvolvimento

O ciclo é iniciado a partir de blocos de requisitos e não a partir de todos os requisitos, como pregado no modelo clássico. A partir daí, a metodologia é a mesma, ou seja, cada atividade somente é iniciada depois que a predecessora é finalizada e validada. Cada bloco de requisitos, quando implantado, corresponde a uma parte do sistema de software, também chamado de incremento. O ciclo é repetido tantas vezes quanto necessário

Prototipagem

Em Engenharia de Software podemos pensar que o protótipo é a primeira versão de um sistema de software.

Esse protótipo de software tem como objetivo a prova de conceitos. Verificar e demonstrar se os requisitos mapeados estão de acordo com a necessidade do usuário ou validar uma solução de projeto.

O modelo de prototipagem possui quatro atividades: • planejamento; • definição das funcionalidades; • construção; • validação.

Quando usar a prototipação?

- Se nosso usuário tem uma boa noção, mas não tem, no momento, uma visão detalhada do que precisa ser feito.
- O cliente precisa ter a total noção de que o protótipo não é o sistema de software.
- Toda a equipe de desenvolvimento também precisa ter a total noção que o protótipo não é o sistema de software

Protótipo de baixa fidelização, que dá ênfase na interação com o usuário e na estrutura geral do sistema de software, não se preocupando em fazer protótipos que produzissem uma imagem real do sistema.

Modelo espiral

Tem como raiz o modelo iterativo incremental e como preocupação central a mitigação de riscos.

A diferença fundamental do modelo espiral para os outros modelos que discutimos até agora é que cada ciclo completo, ou cada iteração, não produz ou implementa um sistema ou uma parte do sistema de software.

O modelo espiral é composto de quatro fases: • Definição dos objetivos. • Mitigação dos riscos. • Desenvolvimento do produto. • Planejamento da próxima fase.

Processo unificado

Processo Unificado, ou UP (Unified Process), tem em seu fundamento a estrutura do modelo iterativo incremental, quatro fases:

- **Iniciação:** nesta fase é definido o escopo do projeto.
- **Elaboração:** o objetivo desta fase é detalhar o escopo, produzindo o modelo de requisitos do sistema. Nesta fase pode-se produzir o plano de projeto, ou plano de desenvolvimento, que consiste no mapeamento dos requisitos, as soluções de software.
- **Construção:** esta fase está associada ao desenvolvimento do projeto, que consiste em projetar, codificar e testar o sistema de software.

- **Transição:** nesta fase, o sistema de software é entregue para o usuário final, sendo comum que existam correções e adaptações a serem feitas.

UP é baseado em três pilares: • Dirigido por casos de uso. • Centrado na arquitetura. • Iterativo e Incremental.

O RUP é um modelo de processos, desenvolvido pela Rational, que associou o Modelo de Processo Unificado às melhores práticas da Engenharia de Software, que são: • Desenvolva iterativamente. • Gerencie requisitos. • Use arquitetura de componentes. • Modele visualmente. • Verifique continuamente a qualidade. • Gerencie mudanças.

Processo x pessoas

- O que será produzido: qual é o produto ou o artefato do processo.
- Como: atividade, ou sequência de atividades, a serem realizadas, em uma determinada sequência para produção do artefato do processo.
- Quando: definição de qual momento o artefato será produzido, definição das pré-condições e pós-condições.
- Quem: indicação do responsável pela atividade.

Papel: “define o comportamento e as responsabilidades de um indivíduo ou de um conjunto de indivíduos que trabalham juntos como uma equipe, no contexto de uma organização de engenharia de software”

No caso do RUP, os papéis são sistematicamente definidos, por exemplo: o analista de sistemas é responsável, entre outras coisas, por desenvolver o plano de gerenciamento de requisitos e o arquiteto de software executa, entre outras atividades, a estruturação do modelo de implantação.

Gerente de projetos

Profissional responsável por gerir o projeto, que compreende, entre muitas coisas, coordenar e acompanhar as atividades do projeto e alocar as pessoas para as funções determinadas.

Analista

É responsável por captar as necessidades do negócio e transformá-las em requisitos do sistema de software.

Projetista

Responsável por produzir uma solução computacional para os problemas identificados. Por exemplo: um projetista de interface gráfica do usuário, que é

especialista em fornecer soluções para interfaces padrão para sistemas operacionais da Apple ou da Microsoft.

Arquiteto de software

Responsável pela elaboração da arquitetura do sistema. Como e quais serão os componentes internos do sistema de software e como esses componentes irão se comunicar para atender a uma determinada necessidade.

Desenvolvedores

Também chamados de programadores, os desenvolvedores têm a responsabilidade de implementar o projeto.

Clientes

O usuário é parte fundamental do processo de desenvolvimento, ele precisa estar envolvido e com foco desde o momento do levantamento de requisitos até a validação.

Avaliadores de qualidade

São responsáveis por garantir que o sistema de software atenda a todas as necessidades do negócio.

O paradigma da orientação a objetos

-paradigma: “Um paradigma é um conjunto de regras que estabelecem fronteiras entre o que é certo e errado, entre o que é verdadeiro e o que é falso, entre o que se deve fazer e o que não se deve fazer [...] Ele funciona como um conjunto de categorias que define e orienta o comportamento das pessoas”

O paradigma da orientação a objetos é uma forma de se desenvolver um sistema de software que o enxerga como um conjunto de componentes que interagem entre si para resolver um determinado problema. A cada componente, dá-se o nome de objeto.

Em orientação a objetos, cada objeto também possui características, denominadas atributos, e a ação a ser executada por esse objeto, que tem o nome de método.

O paradigma da orientação a objetos é baseado nos seguintes pilares: • classes; • objeto; • abstração; • encapsulamento; • herança; • polimorfismo; • ligação, mensagem.

Classe

Classe de objetos pode ser definida como um grupo de objetos com as mesmas propriedades (atributos), comportamentos (operações), relacionamentos e semântica.

Objeto

objeto é um elemento do mundo real, que possui relevância para solução de um determinado problema e que esse objeto possui características e executa determinadas ações (métodos). Esse conjunto de informações, também chamado de atributos, identifica um objeto em uma determinada classe de objetos.

Abstração

O conceito de abstração está ligado à nossa capacidade, enquanto analistas, de resolver problemas, de selecionar determinados aspectos do problema e isolar o que é importante do que não é para um determinado propósito. Abstração é dar ênfase àquilo que é essencial.

Encapsulamento

Encapsulamento significa deixar visível apenas o que é necessário para a comunicação entre dois objetos (Ex: detalhes da implementação ou a lógica algorítmica de um método).

Herança

Objetos que possuem características semelhantes, mas que estão em classes diferentes

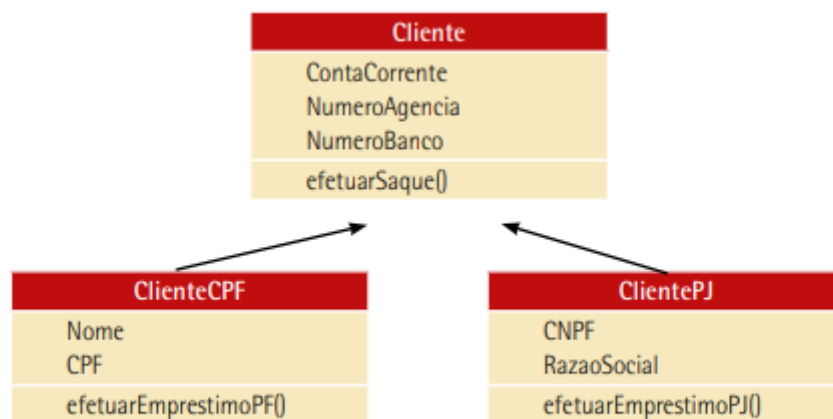


Figura 7 – Exemplo de herança

Dois tipos de herança:

- Simples: quando uma classe filha herda características de apenas uma classe mãe.
- Composta: quando uma classe filha herda características de mais de uma classe mãe.

Polimorfismo

Polimorfismo é quando um objeto tem um comportamento diferente para a mesma ação.

Em orientação a objetos, polimorfismo significa que um método definido na classe mãe é redefinido, tem uma nova implementação, na classe filha, sendo obrigatória a mesma assinatura para o método.

Ligação e mensagem

Mensagem é um meio de comunicação entre objetos com o objetivo de ativar um processamento. Para que um objeto execute um método.

Se um objeto envia uma mensagem para outro, eles possuem uma ligação, chamada de associação simples, ou seja, uma ligação que acontece apenas em um determinado momento.

Pode acontecer que um objeto tenha em sua composição outro objeto. Neste caso existe uma relação todo-parte.

Quando o objeto que compõe outro não existe fora desse contexto, dizemos que existe uma associação do tipo composição.

Quando o objeto que compõe o outro existe fora do contexto, dizemos que há uma associação do tipo agregação.

A linguagem de modelagem unificada

A Linguagem de Modelagem Unificada, ou Unified Modeling Language, conhecida como UML, é uma linguagem para modelar sistemas orientados a objetos.

- **Regra de sintaxe:** cada elemento deve ser desenhado, representado, seguindo um padrão.
- **Regra semântica:** define o significado de cada elemento dentro de um contexto.

UML é uma ferramenta que dá suporte a todo o processo, desde a análise de requisitos, passando pelo projeto, construção e até a validação, um sistema de software pode ser dividido em cinco visões:

- **Caso de Uso:** representa o sistema de um ponto de vista externo, como ele interage com agentes externos, como usuários ou outros sistemas.
- **Projeto:** representa a arquitetura do sistema de software.
- **Implementação:** representa os componentes e subsistemas.
- **Implantação:** representa a distribuição física do sistema e seus componentes e como estes irão se comunicar.

- Processo: dá ênfase na representação do paralelismo e sincronização dos componentes do sistema.

Um diagrama UML é uma composição de elementos gráficos com o objetivo de representar alguma perspectiva do sistema.

ENGENHARIA DE REQUISITOS E MODELOS DE CASOS DE USO

Introdução à Engenharia de Requisitos

Engenharia é um conjunto de métodos, procedimentos e ferramentas que tem por objetivo resolver um determinado problema.

Requisitos são serviços que um sistema deve prestar e suas restrições de funcionamento, refletem as necessidades do cliente.

O mais interessado nos requisitos é o cliente e o usuário, mas eles possuem papéis diferentes. O cliente é o contratante do projeto, o usuário é quem vai, efetivamente, usar o sistema, quem detém o maior conhecimento do negócio, tbm são considerados interessados pelos requisitos: analista de sistemas, desenvolvedores, arquitetos e gerentes de projeto.

Classificação de requisitos

- **Requisitos de usuário:** “declarações em linguagem natural com diagramas de quais serviços o sistema deverá fornecer a seus usuários e as restrições com as quais ele deve operar”. (direcionados aos clientes, usuários, gerentes de projeto e arquitetos de sistema)
- **Requisitos de sistema:** “descrições mais detalhadas das funções, serviços e restrições operacionais do sistema de software”. (direcionados a usuários, arquitetos de sistema e desenvolvedores, pois podem definir uma sequência de implementação)

requisitos não devem contemplar informações de gestão do projeto, detalhes da arquitetura, projeto ou implementação, bem como informações sobre como o sistema será validado. Os requisitos de software devem dar ênfase somente no comportamento do software que será construído e mantido.

Requisitos funcionais

Requisitos funcionais (RF) descrevem o comportamento esperado de um sistema de software, explicita o que o sistema deve fazer e idealmente o que o sistema não deve fazer, requisito funcional é uma descrição detalhada da interação entre o software e o usuário.

Requisitos não funcionais

Requisitos não funcionais (RNF) descrevem restrições sobre os serviços oferecidos pelo sistema de software

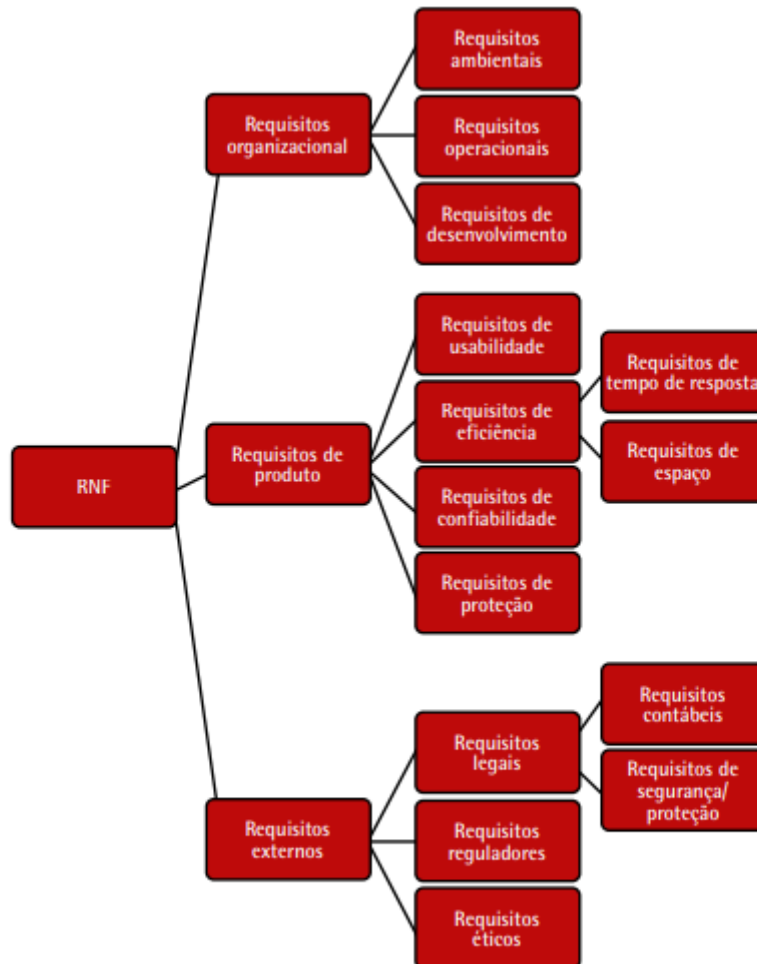


Figura 11 – Requisitos não funcionais

• **Requisitos Organizacionais:** provenientes da organização do cliente, como a padronização de uma linguagem de desenvolvimento. • **Requisitos de Produto:** são aqueles que restringem o comportamento do sistema de software, como o espaço em disco que ele irá ocupar. • **Requisitos Externos:** são requisitos de fora da fronteira do sistema de software, como requisitos legais, de cumprimento da legislação.

Norma ISO/IEC 25010:2011 => descreve seis características que definem a qualidade de software: funcionalidade, confiabilidade, usabilidade, eficiência, manutenibilidade e portabilidade. (atributos de qualidade)

Atributo	Descrição
Funcionalidade	Está ligado à capacidade do sistema de <i>software</i> de prover funcionalidades que atendam às necessidades explícitas e implícitas quando usado sob as condições especificadas
Confiabilidade	Está ligado à capacidade do sistema de <i>software</i> de manter um determinado nível de desempenho quando usado sob as condições especificadas
Usabilidade	Está ligado à capacidade do sistema de <i>software</i> de auxiliar os usuários na realização de suas tarefas, de maneira produtiva (Rocha; Baranauskas, 2003)
Eficiência	Está ligado à capacidade do sistema de <i>software</i> de prover desempenho apropriado, relativo à quantidade de recursos utilizados
Manutenibilidade	Está ligado à capacidade do sistema de <i>software</i> de ser modificado, sendo que essa modificação pode ser uma correção, melhoria ou adaptação
Portabilidade	Está ligado à capacidade do sistema de <i>software</i> de ser portátil entre plataformas de ambientes

Processo de Engenharias de Requisitos

Engenharia de Requisitos é um conjunto de métodos, procedimentos e ferramentas que tem por objetivo descobrir, analisar, documentar, verificar e validar esses requisitos.

O processo de engenharia de requisitos tem como objetivo obter requisitos definidos, especificados e modelos de sistema a partir de fontes de requisitos

As fontes de requisitos podem ser:

- Sistemas de informações existentes.
- Necessidade dos interessados (stakeholders).
- Padrões da organização.
- Informações do Domínio.
- Regulamentos (ou legislações).

Cinco atividades principais:

- elicitação (descoberta dos requisitos a partir das fontes de requisitos),
- análise e negociação (os requisitos são analisados e os conflitos resolvidos por meio da negociação com os interessados. Nessa atividade é produzido o Modelo de Sistema com os requisitos do usuário.),
- documentação (os requisitos são detalhados de tal modo que permitam a realização das próximas atividades do desenvolvimento. É produzida a especificação dos requisitos contendo os requisitos de usuário e de sistema.),
- validação (os requisitos são validados de acordo com os critérios definidos na documentação dos requisitos.)
- gerenciamento (atividade de gerenciamento de requisitos, como se pode notar, não está representada na figura, isso por que a meta da gerência de requisitos é o controle da mudança dos requisitos ao longo do processo da engenharia de requisitos.)

Elicitação de requisitos

A elicitação de requisitos tem como objetivo a descoberta dos requisitos e das fronteiras do sistema, que determinam o escopo do projeto.

Entrevistas

Trata-se basicamente da conversa direcionada a um objetivo: o de obter informações sobre um problema de um entrevistado.

Cenários

Nesta técnica é montado um cenário de interações entre o usuário e o sistema para que ele execute uma determinada tarefa.

Protótipos

Um protótipo é uma versão não operacional do sistema de software que permite esclarecer eventuais requisitos.

Reuniões facilitadas

brainstorming, no qual um grupo de usuários é reunido para debater a respeito de um determinado requisito. É importante que essa reunião ocorra com um mediador, que define o objetivo a ser tratado, capte as principais ideias

Observação

Os analistas aprendem sobre as tarefas do usuário por imersão em seu e observando como eles interagem com seu software e uns com os outros.

Análise de documentos

Os requisitos são elicitados a partir de documentos da organização. Um exemplo muito próximo à nossa realidade são os editais de licitação para desenvolvimento de software

Análise de requisitos

Com os requisitos identificados, é necessário explicitar as informações obtidas na elicitação. Esse processo auxilia no entendimento dos requisitos elicitados, eliminando possíveis ambiguidades e problemas que possam ter passado despercebidos pelo processo de elicitação.

Explicitar os requisitos significa especificar as características operacionais do software, descrever suas restrições e suas interações com outros sistemas a fim de estabelecer um modelo conceitual do problema.

Documentação de requisitos

A documentação dos requisitos tem por objetivo formalizar os requisitos e modelos definidos nas etapas anteriores.

- Documento de definição de sistema: este documento contém os requisitos de sistema, ou seja, requisitos em alto nível de abstração na perspectiva do domínio. Podendo incluir requisitos não funcionais e modelos conceituais.
- Especificação de requisitos de sistema: este documento contém os requisitos na visão sistêmica, ou seja, não somente os requisitos do sistema de software, mas tudo que o circunda.
- Especificação de requisitos de software: são os requisitos na visão do sistema de software. Esse documento pode ser utilizado como base para o desenvolvimento e para as futuras validações.

Validação de requisitos

O objetivo do processo de validação é assegurar que o trabalho de elicitação, análise e documentação dos requisitos está consistente com o domínio do projeto.

Gerenciamento de requisitos

Inevitavelmente, requisitos podem ser incompletos e inconsistentes, bem como novos requisitos podem surgir.

A meta da gerência de requisitos é o controle da mudança dos requisitos ao longo do processo de engenharia de requisitos.

Gerência de requisitos tem mais dois objetivos: • Gerenciar o relacionamento entre requisitos. • Gerenciar o relacionamento entre requisitos e os documentos produzidos durante todo o projeto.

Dois dos modelos mais difundidos atualmente são: CMMI (Capability Maturity Model for Software) e o MPS.BR (Melhoria de Processo do Software Brasileiro).

Controle de versão

É importante que tenhamos sob controle, por qualquer mecanismo que seja, quais documentos fazem parte de uma versão base, uma vez que, a partir da mudança de um requisito, é gerada uma nova versão. Sem um controle de versão, seria praticamente impossível garantir o trabalho em equipe e manter o histórico de evolução do software.

Rastreabilidade

Rastreabilidade especifica as dependências de um requisito para com outro e de um requisito com cada artefato gerado ao longo do projeto. Quando falamos de

artefatos, não estamos falando apenas dos gerados durante a engenharia de requisitos, mas sim em todo o projeto, como qual requisito está relacionado à qual solução de software

Trilha de auditoria => permite identificar tudo que pode ser afetado no caso da mudança de um requisito

Matriz de rastreabilidade => objetivo cruzar um requisito com outro dependente ou um requisito com um artefato.

Existem diversas matrizes de requisitos. Entre as mais utilizadas

- Matriz que indica o relacionamento requisito x requisito.
- Matriz que indica o relacionamento requisito x fonte do requisito.
- Matriz que indica o relacionamento requisito x subsistemas, que descreve quais subsistemas

atendem àquele requisito.

- Matriz que indica o relacionamento entre um requisito elicitado e um artefato produzido no processo da Engenharia de Software

MODELAGEM DE CASOS DE USO E MODELAGEM DE PROCESSO DE NEGÓCIO

Modelagem de casos de uso

Engenharia de requisitos é um conjunto de métodos, procedimentos e ferramentas que tem por objetivo descobrir, analisar, documentar, verificar e validar esses requisitos.

A análise de requisitos tem como objetivo explicitar os requisitos elicitados, o que significa especificar as características operacionais do software.

“modelo de casos de uso é uma representação das funcionalidades externamente observáveis do sistema e dos elementos externos ao sistema que interagem com ele”.

Um caso de uso é a descrição de uma determinada ação ou comportamento de um sistema que produz um resultado para um determinado agente externo a esse sistema, denominado ator.

(Modelagem de caso de uso é um método, aplicável a qualquer modelo de processo de engenharia de requisitos, e UML é uma ferramenta de apoio).

Casos de uso e atores

Caso de uso é a descrição de uma sequência de atividades executadas por um agente externo ao sistema sem que sejam revelados detalhes do funcionamento interno ao sistema.

Atores são os agentes externos ao sistema que executam uma determinada ação e que esperam algum resultado.

- Ator é qualquer entidade externa que interage com o sistema: usuários, outros sistemas de software ou até mesmo dispositivos de hardware.
- Um ator nunca é um componente interno do sistema, ou mesmo qualquer detalhe interno ou componente de implementação dele.
- Um sistema nunca é um ator de si mesmo.

Descrição de casos de uso

Podemos considerar a descrição em linguagem natural, desde que sequencial, como uma descrição de caso de uso

- **Identificação:** nome do caso de uso.
- **Escopo:** descreve sucintamente a que contexto se refere o caso de uso, podendo ser um processo de negócio, um sistema ou um subsistema.
- **Descrição do propósito:** descrição sucinta do objetivo do caso de uso.
- **Ator primário:** ator principal do caso de uso.
- **Pré-condições e pós-condições:** o que deve ser verdadeiro antes e após a finalização do caso de uso.
- **Fluxo normal:** descrição do passo a passo normal do caso de uso.
- **Fluxo alternativo:** descrição de passos alternativos que podem ser executados dentro de uma sequência normal de passos.
- **Requisitos relacionados:** lista de requisitos que estão representados pelo caso de uso. Um caso de uso pode conectar um ou mais requisitos de tipos diferentes, funcionais e não funcionais.

Diagrama de caso de uso

Diagrama de casos de uso é um diagrama da UML que tem por objetivo mostrar, a partir de um ponto de vista estático, o conjunto de casos de uso, atores e seus relacionamentos. Visão estática é o termo usado para descrever uma parte do sistema sob o ponto de vista estrutural.

Estrutura do diagrama de caso de uso

Relacionamento entre casos de uso

Três tipos de relacionamento entre casos de uso: inclusão, extensão e herança.

Inclusão

Significa que o comportamento definido no caso de uso de inclusão é incorporado ao comportamento do caso de uso base, ou seja, para que este seja executado, obrigatoriamente o caso de uso de inclusão também deverá ser executado. Na notação da UML, um relacionamento de inclusão entre casos de uso é mostrado como uma dependência (seta pontilhada) estereotipada com a palavra-chave include.

Extensão

Significa que o comportamento definido no caso de uso de extensão pode ou não ser incorporado ao comportamento do caso de uso base, ou seja, para que este seja executado, o caso de uso de extensão pode ou não ser executado, notação da UML, um relacionamento de extensão entre casos de uso é mostrado como uma dependência (seta pontilhada) estereotipada com a palavra-chave extend.

Herança

Significa que o caso de uso filho herda o comportamento e o significado do caso de uso pai, acrescentando ou mudando seu comportamento, notação da UML, um relacionamento de herança entre casos de uso é mostrado com uma linha cheia com uma seta aberta.

Podemos concluir que os casos de uso filho são especializações se, e somente se, mantiverem o mesmo objetivo (filho é uma especialização do caso de uso pai, ou o pai é uma generalização do caso de uso filho)

Relacionamento entre atores

Atores possuem apenas um tipo de relacionamento: herança. Significa que o ator filho herda o comportamento e o significado do ator pai, acrescentando ou mudando seu comportamento, um relacionamento de herança entre atores é a mesma notação utilizada para cenários de uso: uma linha cheia com uma seta aberta (UML). Ator Filho é uma especialização do Ator Pai, ou que o Ator Pai é uma generalização do Ator Filho.

Modelagem de processos de negócio

Introdução à modelagem de processos de negócio

Processos de negócio são atividades relacionadas a um determinado negócio, que são executadas em uma determinada sequência e que produzem um determinado resultado ou objetivo

• Why/Por que/Motivo: representa o objetivo do processo. • Where/Onde: representa onde o processo (a atividade) é executado, as características do ambiente. Podemos representar uma unidade da organização ou até mesmo uma localização geográfica. • When/Quando: representa o momento em que a atividade é executada e suas restrições temporais. • Who/Quem: representa quem é o responsável ou os envolvidos na execução da atividade. Se fizermos uma analogia ao modelo de caso de uso, são os nossos atores. • What/O que: representa um conjunto de informações que serão utilizadas na execução da atividade, são os artefatos que serão produzidos e/ou consumidos na atividade. • How/Como: é a junção de todos os 5W's que definem como as atividades serão executadas. parei na 33

O papel do analista de negócio

O analista de negócio é o profissional responsável por captar as reais necessidades dos stakeholders e não apenas seus desejos expressos. Ele estabelece um elo entre o usuário do sistema de informação e o sistema de informação propriamente dito, estabelece a ligação entre a área de negócio e a área de TI

O analista de negócio atua nas seguintes áreas de conhecimento: • análise de requisitos; • análise corporativa; • avaliação e validação da solução; • elicitação; • gestão e comunicação de requisitos; • planejamento e monitoramento da análise de negócio

Regras de negócio

Elas são um conjunto de restrições que definem como um processo de negócio de uma organização deve ser executado, que, além de representar determinados conhecimentos a respeito de um processo, também representam importantes aspectos restritivos na execução deste processo

exemplos: • O cliente não pode efetuar saques no período entre 23h00min e 05h00min. • O cliente pode efetuar um saque de, no máximo, R\$300,00 em uma única operação. • O limite de saques diários de um cliente é de R\$1000,00. • Caso o cliente efetue um saque com opção de contratação de empréstimo, serão cobrados juros de 10% ao mês.

UML

Diagrama de processo

• Recursos: representam os objetos de uma organização, como pessoas, materiais e informações, que são usados, consumidos refinados ou produzidos em um processo de negócio. • Processos: representam atividades executadas em uma organização. • Objetivos: representam o resultado que se deseja alcançar, as metas da organização. • Regras: representam as particularidades que restringem algum aspecto do negócio e representam conhecimento do negócio. • Eventos: representam a mudança de estado do negócio. Um evento pode ser gerado por um processo, que, inclusive, pode estar fora do negócio e é recebido por um ou mais processos.

Processo

O símbolo que representa um processo pode possuir ou não a indicação do estereótipo <>, mas obrigatoriamente deverá ser representada a descrição textual do processo.

Objeto

de negócio Um objeto de negócio é a representação de uma entrada ou saída de um processo. Em geral, são utilizados estereótipos para diferenciar e explicitar melhor os tipos de objeto de negócio, que são: <>

Objeto de informação

Objeto que identifica informações produzidas ou consumidas pelo processo

Dependência

A conexão utilizando linha tracejada e seta indica dependência entre os componentes do modelo, sendo que a direção da seta indica a direção da dependência.

diagramas utilizados na modelagem de processos de negócio: • Diagrama de caso de uso. • Diagrama de modelo de processo. • Diagrama de atividade.

Diagrama de atividade

Definido na UML, o diagrama de atividade representa um fluxo de atividades que tem como objetivo atingir um determinado objetivo.

Adornos da UML => Adornos são mecanismos ou elementos da linguagem UML utilizados com o único intuito de facilitar o entendimento ou ratificar o significado de algo que estamos querendo representar.